

Chase Coleman
CSE 511: Hot Spot Analysis Project Report
ASU ID: 1239783392
6/17/2026

Reflection

For this project, I worked on two different spatial analysis tasks in Spark: hot zone analysis and hot cell analysis. I started by reading through the provided template code and figuring out which parts were already handled and which parts still needed to be implemented. The template already took care of a lot of the setup, like reading input files and parsing command line arguments, so most of my work was focused on the actual spatial logic and the Spark transformations.

The hot zone analysis was the more straightforward part of the project. I needed to determine whether a pickup point was inside a given rectangle. Once that function worked, the rest of the task was mostly joining the point data with the rectangle data, counting how many points belonged to each rectangle, and sorting the final output. One thing I had to be careful about was matching the expected output exactly, including sorting and how duplicate rectangles were handled.

The hot cell analysis was more involved. Instead of just checking whether a point was inside a rectangle, I had to group taxi pickups into spatial and temporal cells, then calculate which cells were statistically significant hot spots. I broke the problem down into smaller steps: convert each pickup into x, y, and z coordinates, count the number of pickups in each cell, compute the overall mean and standard deviation, calculate each cell's neighborhood sum, and then use the Getis-Ord formula to rank the cells.

Testing was a big part of the process. I built the jar with `sbt clean assembly`, ran it with `spark-submit`, and compared the output against the sample answers. I also ran into a version issue where my code worked locally but failed in the autograder because the grading environment used a newer Spark version. Fixing the Scala and Spark versions in the build file was an important reminder that code can be logically correct but still fail if the runtime environment does not match.

Lessons Learned

This project helped me understand how spatial analysis can be done at scale. The basic idea of checking whether points fall inside a region is simple, but doing it through Spark made me think more about how data is transformed, grouped, and aggregated across a larger dataset.

I also got more comfortable with Spark DataFrames and SQL-style operations. The hot zone task helped reinforce joins, grouping, counting, and sorting. The hot cell task pushed me further because it required both data processing and a statistical calculation. I had to think carefully

about the full grid of cells, including cells with zero pickups, because the Getis-Ord statistic depends on the entire set of possible cells.

Another thing I learned was how important small implementation details can be. For example, sorting the output correctly matters, preserving duplicate rectangles matters, and using the right Scala/Spark version matters. These are not complicated ideas by themselves, but missing one of them can cause the whole submission to fail.

Implementation

For hot zone analysis, the main function I implemented was:

```
def ST_Contains(queryRectangle: String, pointString: String): Boolean
```

This function checks whether a point is inside a rectangle. The rectangle is given as four numbers: two longitude values and two latitude values. The point is given as a longitude and latitude pair. I parsed the strings into numbers, found the rectangle's minimum and maximum x and y values, and then checked whether the point's coordinates fell inside those limits.

After that, I used the containment function in a Spark SQL join. Each rectangle was matched with the pickup points inside it. Then I grouped the results by rectangle and counted the number of matching points. The final result was sorted by the rectangle string so that the output matched the required format.

For hot cell analysis, I first converted each taxi pickup into a cell coordinate. The x value came from longitude, the y value came from latitude, and the z value came from the day of the month. The template already provided the basic coordinate conversion, so I used that to create (x, y, z) values for each pickup.

Next, I counted how many pickups were in each cell. I also generated the full set of possible cells in the required coordinate range, because cells with zero pickups still matter when calculating the overall mean and standard deviation. After that, I calculated the total number of cells, the average number of pickups per cell, and the standard deviation.

To calculate the hotness of each cell, I found the sum of pickup counts in the surrounding neighborhood of each cell. This included nearby cells in the x, y, and z directions. I also handled edge cases where a cell was on the boundary and had fewer neighbors.

Finally, I used the Getis-Ord formula to calculate a G-score for each cell. The output was sorted from highest G-score to lowest G-score, and only the cell coordinates were returned, since the instructions said not to include the score itself.

Overall, this project gave me practice combining spatial logic, Spark transformations, and statistical analysis. It also showed me how important it is to test carefully and make sure the final output matches the assignment format exactly.